UNIT 3 · TOPIC 3.5

Network Threat Detection

Assignment, Answer Key

Teacher answer key, 32 points total.

Practice Questions

3.5.a · Intrusion detection and prevention systems (IDS/IPS)

1. An IDS raises an alert on a packet that turns out to be a legitimate network scanner run by the IT team. This outcome is called a: (1 pt)

- ○ A. True positive
- **● B. False positive**
- ○ C. False negative
- ○ D. True negative

Why: The sensor fired (positive) but the traffic was benign (false). That is a false positive, wasted analyst effort but no missed attack.

2. Which statement BEST distinguishes an IPS from an IDS? (1 pt)

- ○ A. An IPS uses anomaly detection; an IDS uses signatures.
- ○ B. An IDS monitors host logs; an IPS monitors network traffic.
- **● C. An IPS can drop malicious packets in real time; an IDS only generates alerts.**
- ○ D. An IPS is deployed outside the firewall; an IDS is deployed inside.

Why: The defining difference is action: IDS is passive (alerts only); IPS is inline and can block traffic before it reaches the target.

3. Select ALL true statements about signature-based detection. (3 pts)

- □ A. It can detect zero-day exploits that have no known signature.
- **■ B. It requires the signature database to be kept current.**
- **■ C. It matches traffic against patterns of known attacks.**
- **■ D. It typically has a lower false-positive rate than anomaly detection for known attacks.**

Why: Signatures must be updated (1), work by pattern matching (2), and are precise for known attacks (3). Zero-days have no signature, so option 0 is false.

4. A real attack passes through the IDS without triggering any alert. This is called a: (1 pt)

- **● A. False negative**
- ○ B. False positive
- ○ C. True negative
- ○ D. True positive

Why: The sensor said benign (negative) but the traffic was an attack (false), a false negative, the most dangerous outcome.

5. Name the detection method that builds a behavioral baseline of normal activity and alerts on deviations, even for attacks with no known signature. (1 pt)

Acceptable answers: anomaly; anomaly-based; anomaly based; behavior; behavioral; heuristic

Why: Anomaly-based (behavioral) detection learns a normal baseline and flags statistical deviations, making it capable of catching novel attacks.

3.5.b · Network monitoring and traffic analysis

6. Which data source summarizes a conversation (endpoints, ports, protocol, byte counts) WITHOUT storing packet payload? (1 pt)

- ● **A. A flow record (NetFlow)**
- ○ B. Full packet capture (pcap)
- ○ C. A signature database
- ○ D. A connection-tracking table

Why: Flow records summarize a conversation's metadata without payload, making them far cheaper to store and query at scale than full packet capture.

7. A host normally sends a few MB per day and suddenly uploads several GB to an external server overnight. This is a classic indicator of: (1 pt)

- ○ A. Beaconing
- ○ B. Rule shadowing
- ● **C. Data exfiltration**
- ○ D. A false positive

Why: An abnormal spike in outbound data volume, especially to an external destination, is a strong exfiltration indicator, volume alone is the signal, no payload inspection required.

8. Malware checking in with a command-and-control server at exact 60-second intervals is exhibiting: (1 pt)

- ○ A. Baselining
- ● **B. Beaconing**
- ○ C. Rule shadowing
- ○ D. A true negative

Why: Regular, machine-precise timing (as opposed to irregular human-driven traffic) is the hallmark of beaconing malware.

9. Select ALL indicators that traffic analysis (not signature matching) is well suited to catch. (3 pts)

- ■ **A. Unusual destination port for a host**
- □ B. A known malware file hash
- ■ **C. A data-volume spike outside business hours**
- ■ **D. Beaconing at regular intervals**

Why: Port anomalies, volume spikes, and beaconing are all traffic-pattern indicators. A known file hash match is signature-based detection, not traffic analysis.

10. Name the process of recording a host's typical traffic pattern (volume, ports, timing) so deviations can later be detected. (1 pt)

Acceptable answers: baselining; baseline; establishing a baseline

Why: Baselining establishes the 'normal' profile that anomaly detection compares current traffic against.

11. Why do most enterprise monitoring pipelines rely on flow records rather than full packet capture for day-to-day analysis? (1 pt)

- ○ A. Flow records are more secure than packet captures
- ● **B. Flow records are far cheaper to store and query at scale**

○ C. Packet capture cannot see TCP ports

○ D. Flow records include the full payload, which is more useful

Why: Full packet capture is precise but storage- and search-expensive at scale; flow records trade payload detail for cost efficiency, which suits continuous monitoring.

3.5.c · Incident response sequence

12. Which incident response phase involves isolating an infected host from the network WITHOUT yet removing the malware? (1 pt)

○ A. Identification

○ B. Eradication

● **C. Containment**

○ D. Recovery

Why: Containment stops the spread (isolate the host, block the attacker's address) while preserving evidence; eradication then removes the malware and closes the hole. NIST SP 800-61 groups containment, eradication and recovery into one phase, but they are still done in that order.

13. Put in order: which phase comes immediately AFTER Containment? (1 pt)

● **A. Eradication**

○ B. Identification

○ C. Preparation

○ D. Lessons Learned

Why: The canonical order is Preparation, Identification, Containment, Eradication, Recovery, Lessons Learned, Eradication follows Containment.

14. Select ALL reasons eradicating malware BEFORE containment is a mistake. (3 pts)

■ **A. It can tip off the attacker before the host is isolated**

■ **B. The host may be immediately reinfected from another compromised host on the same network**

□ C. It skips the Preparation phase entirely

■ **D. It removes the root cause before the spread is stopped**

Why: Eradicating before containing risks tipping off the attacker, reinfection from the still-connected network, and generally removes the cause before the spread is controlled. It has nothing to do with skipping Preparation.

15. A team documents a full incident timeline and updates its response plan afterward. Which phase is this? (1 pt)

● **A. Lessons Learned**

○ B. Eradication

○ C. Recovery

○ D. Identification

Why: Lessons Learned (Post-Incident Activity) is the closing phase: documenting what happened and updating the plan/tools/training.

16. Name the NIST publication that defines the six-phase incident response lifecycle used in this course. (1 pt)

Acceptable answers: nist sp 800-61; 800-61; sp 800-61; nist 800-61

Why: NIST Special Publication 800-61, 'Computer Security Incident Handling Guide,' defines the standard IR lifecycle.

17. Which phase happens continuously BEFORE any incident occurs, and includes writing the IR plan and running tabletop exercises? (1 pt)

- ○ A. Identification
- ● **B. Preparation**
- ○ C. Containment
- ○ D. Recovery

Why: Preparation is ongoing readiness work done before any incident, plans, roles, tooling, and rehearsal.

Coding Assignments (Python 3)

3.5.a · Intrusion detection and prevention systems (IDS/IPS)

3 pts · auto-graded on the site

Coding problem 3.5.a: IDS signature matcher

Skill: Detect Attacks · **Maps to:** 3.5.A · **Language:** Python 3 (pytest)

Problem

You are implementing the rule-matching core of a simple IDS. The sensor receives a stream of packet-log entries. Each entry is a dict with the fields shown below. You are given a list of signature rules; each rule has a destination port and a keyword that must appear (case-insensitive) in the payload. A log entry matches a rule when BOTH the port AND the keyword condition are met.

Return the sorted list of `id` values for every log entry that matches at least one rule.

Function contract

Implement `solve(logs: list[dict], rules: list[dict]) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `logs`: list of dicts, each with keys: - `"id"` (str), unique entry identifier, e.g. `"pkt-001"` - `"dst_port"` (int), destination port of the packet - `"payload"` (str), packet payload (may be empty)
- `rules`: list of dicts, each with keys: - `"port"` (int), destination port to match - `"keyword"` (str), substring to find (case-insensitive) in the payload

Output

A sorted `list[str]` of `id` values for entries that match at least one rule. Empty list if no entries match.

Examples

```
logs = [
    {"id": "pkt-001", "dst_port": 80,  "payload": "GET /admin/../etc/passwd HTTP/1.1"},
    {"id": "pkt-002", "dst_port": 443, "payload": "TLS handshake"},
    {"id": "pkt-003", "dst_port": 80,  "payload": "GET /index.html HTTP/1.1"},
    {"id": "pkt-004", "dst_port": 22,  "payload": "SSH-2.0-OpenSSH_8.9"},
]
rules = [
    {"port": 80, "keyword": "etc/passwd"},
    {"port": 22, "keyword": "libssh"},
]
solve(logs, rules)  ->  ["pkt-001"]
```

```
logs = []
solve(logs, rules)  ->  []
```

Constraints

- Deterministic, offline, standard library only.
- Keyword matching is case-insensitive substring match.
- A log entry that matches multiple rules is included only once in the output.
- Output is sorted ascending by `id`.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from typing import List, Dict, Any


def solve(logs: List[Dict[str, Any]], rules: List[Dict[str, Any]]) -> List[str]:
    matched = set()
    for entry in logs:
        port = entry["dst_port"]
        payload_lower = entry["payload"].lower()
        for rule in rules:
            if rule["port"] == port and rule["keyword"].lower() in payload_lower:
                matched.add(entry["id"])
                break
    return sorted(matched)
```

3.5.b · Network monitoring and traffic analysis

3 pts · auto-graded on the site

Coding problem 3.5.b: Traffic-anomaly detector

Skill: Detect Attacks · **Maps to:** 3.5.B · **Language:** Python 3 (pytest)

Problem

Implement a simplified network traffic-anomaly detector. You are given a per-host **baseline** (the normal profile for that host) and a window of **observed flow records**. Flag every host that either exceeds its baseline byte-volume threshold or uses a destination port not in its baseline's allowed-port set.

A **baseline** is a dict mapping host IP (str) to a dict with:

- `"max_bytes"`: integer, the maximum total outbound bytes this host normally sends in the window
- `"allowed_ports"`: list of integers, destination ports this host normally uses

An **observed flow record** is a dict with:

- `"host"`: source host IP string
- `"dst_port"`: destination port integer
- `"bytes"`: integer, bytes sent in this flow

A host may appear in multiple flow records within the same window; volumes accumulate.

Function contract

Implement `solve(baseline, flows)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

- `baseline`: dict as described above
- `flows`: list of flow record dicts (any order)

Return a sorted `list[str]` of host IPs flagged as anomalous. A host is flagged if EITHER: 1. Its **total** bytes across all its flow records in this window exceeds `max_bytes`, OR 2. Any single flow record for that host uses a `dst_port` not present in `allowed_ports`.

A host not present in `baseline` at all is always flagged (unknown host, no baseline to compare against, treated as an anomaly).

Examples

```
baseline = {
    "10.0.1.5": {"max_bytes": 500_000_000, "allowed_ports": [445, 139]},
    "10.0.2.9": {"max_bytes": 5_000_000, "allowed_ports": [80, 443]},
}
flows = [
    {"host": "10.0.1.5", "dst_port": 445, "bytes": 100_000_000},
    {"host": "10.0.1.5", "dst_port": 445, "bytes": 50_000_000},
    {"host": "10.0.2.9", "dst_port": 4444, "bytes": 1_000},   # unusual port -> flagged
]
solve(baseline, flows) == ["10.0.2.9"]
```

```
baseline = {"10.0.3.1": {"max_bytes": 1_000_000, "allowed_ports": [443]}}
flows = [
    {"host": "10.0.3.1", "dst_port": 443, "bytes": 900_000},
    {"host": "10.0.3.1", "dst_port": 443, "bytes": 900_000},   # total 1.8M > 1M -> flagged
]
solve(baseline, flows) == ["10.0.3.1"]
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).
- Result is sorted, contains no duplicate host IPs.

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from typing import Dict, List


def solve(baseline: Dict, flows: List[Dict]) -> List[str]:
    totals: Dict[str, int] = {}
    port_violation: Dict[str, bool] = {}
    unknown_hosts = set()

    for flow in flows:
        host = flow["host"]
        if host not in baseline:
            unknown_hosts.add(host)
            continue
```

```
        totals[host] = totals.get(host, 0) + flow["bytes"]
        allowed_ports = baseline[host]["allowed_ports"]
        if flow["dst_port"] not in allowed_ports:
            port_violation[host] = True

    flagged = set(unknown_hosts)
    for host, total in totals.items():
        if total > baseline[host]["max_bytes"]:
            flagged.add(host)
    for host in port_violation:
        flagged.add(host)

    return sorted(flagged)
```

3.5.c · Incident response sequence

3 pts · auto-graded on the site

Coding problem 3.5.c: Incident response phase sorter

Skill: Detect Attacks, Collaborate · **Maps to:** 3.5.C · **Language:** Python 3 (pytest)

Problem

The NIST SP 800-61 incident response lifecycle has six phases, in this canonical order:

```
1. Preparation
2. Identification
3. Containment
4. Eradication
5. Recovery
6. Lessons Learned
```

You will implement a function that takes a list of response-action records, each tagged with the phase it belongs to, but given in shuffled order, and returns the actions sorted into correct lifecycle order. Actions within the same phase keep their original relative order (a stable sort).

An **action record** is a dict with:

- `"action"`: string description of the action
- `"phase"`: string, one of the six canonical phase names exactly as spelled above

Function contract

Implement `solve(actions)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

- `actions`: list of action record dicts, in arbitrary order

Return a `list[str]` of the `"action"` text values, reordered so that all actions from an earlier phase (per the canonical order above) come before all actions from a later phase. Within the same phase, preserve the original input order (stable sort). If an action's `"phase"` value is not one of the six canonical names, raise `ValueError`.

Examples

```
actions = [
    {"action": "Rebuild the server from a clean backup", "phase": "Recovery"},
    {"action": "Write the incident response plan", "phase": "Preparation"},
    {"action": "Isolate the infected host from the network", "phase": "Containment"},
    {"action": "Confirm the IDS alert is a real compromise", "phase": "Identification"},
]
solve(actions) == [
    "Write the incident response plan",
    "Confirm the IDS alert is a real compromise",
```

```
    "Isolate the infected host from the network",
    "Rebuild the server from a clean backup",
]

actions = [
    {"action": "Bogus phase", "phase": "Investigation"},
]
solve(actions)  # raises ValueError
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).
- Sort is stable within a phase.

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from typing import Dict, List

PHASE_ORDER = [
    "Preparation",
    "Identification",
    "Containment",
    "Eradication",
    "Recovery",
    "Lessons Learned",
]
_PHASE_RANK = {name: i for i, name in enumerate(PHASE_ORDER)}


def solve(actions: List[Dict]) -> List[str]:
    for entry in actions:
        if entry["phase"] not in _PHASE_RANK:
            raise ValueError(f"Unknown IR phase: {entry['phase']!r}")

    ordered = sorted(actions, key=lambda entry: _PHASE_RANK[entry["phase"]])
    return [entry["action"] for entry in ordered]
```